FROM ZERO TO .NET

A complete, beginner-friendly guidebook to the C# language and the .NET platform — written for programmers who already know Python, C, C++, or Java and want to become productive in C# and .NET quickly.

Prepared for Subhajit
B.Tech CSE, Semester 6 · MAKAUT

Beginner to Advanced · Theory + Code Examples + Practice Exercises

Table of Contents

Part V: Advanced & Modern C#

Part VI: Into the .NET Ecosystem

PART I

Getting Started

Foundations before you write real code

CHAPTER 1

Introduction to C# and .NET

You already know how to program — Python, C, C++, and Java have taught you variables, loops, functions, and objects. This book will not re-teach programming from scratch. Instead, it will teach you C# by constantly relating it to what you already know, while making sure every C#-specific idea is explained clearly with runnable examples and exercises. By the end, you will be ready to build real applications on .NET.

1.1 What is C#?

C# (say "C sharp") is a modern, general-purpose, object-oriented, statically typed programming language created by Microsoft in 2000, designed by Anders Hejlsberg (who also designed Turbo Pascal and Delphi, and later TypeScript). It was built to combine the performance and type-safety of C++ with the productivity and simplicity of languages like Java.

- **Statically typed** — like C, C++, and Java. Types are checked at compile time.
- **Object-oriented** — everything lives inside classes, similar to Java.
- **Garbage collected** — like Python and Java, you don't manually free memory like in C/C++.
- **Compiled** — C# code compiles to an intermediate language (IL), not directly to machine code.
- **Multi-paradigm** — supports OOP, functional-style (LINQ, lambdas), and generic programming.

> **Coming from Python / C / C++ / Java**
> If you know Java, C# will feel extremely familiar — classes, interfaces, garbage collection, single inheritance, and a huge standard library all carry over almost directly.
> If you know C/C++, you'll recognize the syntax (braces, semicolons, similar operators) but you will no longer manage memory manually, and pointers are restricted to special "unsafe" blocks.
> If you know Python, the biggest adjustment is explicit typing and compiling before running, but C# has many features (like `var`, LINQ, and string interpolation) that will feel Python-like in convenience.

1.2 What is .NET?

C# is just a language — a syntax and a set of grammar rules. .NET is the **platform** that actually runs your C# code. Think of the relationship the way Java relates to the JVM, or the way Python code relates to the CPython interpreter.

- **CLR (Common Language Runtime)** — the virtual machine that executes your compiled code. Equivalent to the JVM in the Java world.
- **BCL (Base Class Library)** — the standard library: collections, file I/O, networking, string handling, and more. Equivalent to Java's `java.*` packages or Python's standard library.
- **.NET SDK** — the tools you install to compile and run C# code (compiler + CLI + libraries).

- **NuGet** — the package manager, equivalent to pip (Python) or Maven/Gradle (Java).

Modern .NET (called simply ".NET", version 5 and above, currently .NET 8/9) is free, open-source, and cross-platform — it runs on Windows, Linux, and macOS. This is different from the old ".NET Framework" (pre-2020), which was Windows-only. Whenever this book says ".NET", it means modern, cross-platform .NET.

> **NOTE — Why learn C# and .NET now?**
> C# and .NET power a huge range of software: backend APIs (ASP.NET Core), desktop apps (WPF, MAUI), game development (Unity uses C# as its scripting language), cloud services (Azure), and cross-platform mobile apps. It's one of the most in-demand backend stacks in enterprise software, alongside Java and Node.js.

1.3 How C# Code Runs: Compilation Model

When you build a C# project, the compiler (`csc`, invoked automatically by the `dotnet` CLI) translates your `.cs` source files into **Intermediate Language (IL)** — a CPU-independent bytecode, packaged into a `.dll` or `.exe` assembly. When you run the program, the CLR's **JIT (Just-In-Time) compiler** translates that IL into native machine code for your specific CPU, right before it executes.

Stage	Java equivalent	C#/.NET equivalent
Source code	`.java`	`.cs`
Compiled bytecode	`.class` (bytecode)	`.dll`/`.exe` (IL)
Runtime engine	JVM	CLR
Package manager	Maven / Gradle	NuGet
Standard library	JDK class library	BCL (Base Class Library)

This two-step compilation (source → IL → native code) is exactly how Java works with bytecode and the JVM. It's what gives C# its cross-platform ability: the same compiled `.dll` can run on Windows, Linux, or macOS as long as .NET is installed there.

> **Practice Exercises — Chapter 1**
> 1. In your own words (2-3 sentences), explain the difference between C# and .NET to a friend who only knows Python.
> 2. Name two things C# has in common with Java, and two things it has in common with C++.
> 3. Why can the same compiled C# program run on both Windows and Linux without recompiling? (Hint: think about IL and the CLR.)

CHAPTER 2

Setting Up & Your First Program

Let's get C# installed and write, compile, and run your first real program. Unlike Python, C# code must be compiled before it runs — but the `dotnet` command-line tool makes this almost as smooth as running a Python script.

2.1 Installing the .NET SDK

Download and install the .NET SDK (Software Development Kit) from **dotnet.microsoft.com/download**. Get the latest LTS (Long Term Support) version, currently .NET 8. The SDK includes the compiler, runtime, and the `dotnet` CLI tool.

Verify your installation (any terminal: cmd, PowerShell, or bash)

```
dotnet --version
# e.g. output: 8.0.100

dotnet --info
# shows installed SDKs, runtimes, and OS info
```

NOTE — Editor choice
Visual Studio Code with the "C# Dev Kit" extension is a lightweight, cross-platform choice (similar to using VS Code for Python or Node). Visual Studio (Windows only, full IDE) is heavier but has the richest tooling. Either is fine for this book — all examples use the `dotnet` CLI, which works the same regardless of editor.

2.2 Creating Your First Project

In C#, code doesn't live as a loose `.cs` file the way a `.py` script can run standalone. Instead, files belong to a **project**, described by a `.csproj` file (similar in spirit to a Java `pom.xml`/`build.gradle`, or a Python `pyproject.toml`). The `dotnet` CLI scaffolds this for you.

Create and run a console app

```
dotnet new console -o HelloCSharp
cd HelloCSharp
dotnet run
```

This creates a folder with two important files: **HelloCSharp.csproj** (project metadata: target framework, dependencies) and **Program.cs** (your source code). `dotnet run` compiles and immediately executes the program — similar to how `python script.py` both "compiles" (parses) and runs in one step.

Program.cs (default template, .NET 6+, "top-level statements")

```
// This is the whole program. No visible Main method needed for a simple script!
Console.WriteLine("Hello, World!");
```

NOTE — Top-level statements
Since C# 9 / .NET 6, a single file can skip the class-and-Main boilerplate for quick programs — much like a Python script. Under the hood, the compiler still wraps this in a class with a `Main` method; it's just hidden. Larger, multi-file programs will still define classes explicitly, as you'll see from Chapter 10 onward.

2.3 The Classic (Explicit) Form

It's important to recognize the traditional, explicit form too, since most tutorials, older code, and multi-class projects use it. This is the direct equivalent of Java's `public static void main(String[] args)`.

Program.cs — traditional form

```
using System;

namespace HelloCSharp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Piece	Meaning
`using System;`	Import directive — brings the `System` namespace into scope (like Java's `import` or Python's `import`).
`namespace HelloCSharp`	Groups related classes together, exactly like Java packages.
`class Program`	Every executable statement in C# must live inside a class — there's no bare top-level `def main()` like Python.
`static void Main(string[] args)`	The entry point. `static` means it belongs to the class itself, not an instance. `args` holds command-line arguments, like Java's `String[] args`.
`Console.WriteLine(...)`	Prints a line to standard output — equivalent to `System.out.println` (Java) or `print()` (Python).

2.4 Compiling & Running: What Happens Under the Hood

Useful dotnet CLI commands

```
dotnet build      # compiles the project into a DLL, without running it
dotnet run        # builds (if needed) and runs
dotnet publish -c Release   # produces an optimized, deployable build
dotnet clean      # removes build output
```

Coming from Python / C / C++ / Java

`dotnet run` is the closest analogue to `python script.py` or `java Main.java` (single-command run), but internally it's a real two-stage compile-then-execute process, just automated for you.

Practice Exercises — Chapter 2

1. Install the .NET SDK and confirm `dotnet --version` works on your machine.

2. Create a console project called `Greetings` and modify it to print your name and college on two separate lines.

3. Rewrite your `Greetings` program using the explicit `class Program { static void Main(...) }` form instead of top-level statements.

4. Run `dotnet build` on your project, then look inside the `bin/Debug` folder. What files were generated?

CHAPTER 3

Variables, Data Types & the Type System

C# is statically typed: every variable has a fixed type, checked at compile time — the compiler will refuse to build code with type errors, unlike Python where type errors surface at runtime. This chapter covers C#'s built-in types and how variables work.

3.1 Declaring Variables

Basic variable declarations

```
int age = 21;
double price = 499.99;
char grade = 'A';
bool isPassed = true;
string name = "Subhajit";

// Declare, then assign later
int score;
score = 85;
```

Every variable declaration has the form `type name = value;`. Once declared with a type, a variable can never hold a value of a different, incompatible type — the compiler enforces this.

Type inference with var

Using var — type is still fixed, just inferred

```
var age = 21;          // compiler infers int
var price = 499.99;    // compiler infers double
var name = "Subhajit"; // compiler infers string

// age = "twenty-one";  // COMPILE ERROR — age is still an int under the hood
```

WATCH OUT — var is NOT dynamic typing
This is a common misconception coming from Python. `var` only tells the **compiler** to figure out the type from the right-hand side at compile time. Once inferred, the type is locked in forever for that variable, exactly like an explicit declaration. It is purely a convenience for the programmer, not a feature like Python's dynamic typing.

3.2 Built-in (Primitive) Types

C# type	Size	Range / Notes	Closest in C/C++/Java
`int`	4 bytes	-2,147,483,648 to 2,147,483,647	`int`
`long`	8 bytes	much larger range, suffix with `L`	`long`
`short`	2 bytes	-32,768 to 32,767	`short`
`byte`	1 byte	0 to 255 (unsigned)	`unsigned char`
`double`	8 bytes	64-bit floating point, default for decimals	`double`
`float`	4 bytes	32-bit floating point, suffix with `f`	`float`
`decimal`	16 bytes	High precision, used for money	no direct equivalent
`bool`	1 byte	`true` or `false` (not 0/1 like C)	`bool`
`char`	2 bytes	single UTF-16 character, single quotes	`char`
`string`	reference type	text, double quotes, immutable	`String`/`std::string`

Literals and suffixes

```
long population = 8_000_000_000L;     // L suffix for long
float pi = 3.14159f;                  // f suffix for float
decimal price = 1999.50m;             // m suffix for decimal (money!)
int hex = 0xFF;                       // hexadecimal, = 255
int binary = 0b1010;                  // binary, = 10
int readable = 1_000_000;             // underscores for readability
```

NOTE — Use decimal for money, double for science

`double`/`float` use binary floating point and can introduce small rounding errors (just like Python's `float` or C's `float`/`double`) — never use them for currency. `decimal` is base-10 and precise, designed specifically for financial calculations.

Coming from Python / C / C++ / Java

Java has almost the exact same primitive set (`int`, `long`, `double`, `boolean`, `char`), so this table should feel very familiar. C/C++ programmers should note `bool` is a real, safe type here (not just an int), and there's no unsigned `int` by default — use `uint`/`ulong` explicitly if you need unsigned semantics.

3.3 Value Types vs Reference Types

This is one of the most important concepts in C#, and it trips up people from Python/Java the most. All the primitive types above (`int`, `double`, `bool`, `char`, and `struct`s) are **value types**: a variable directly contains its data, and assigning it **copies** the value. Types like `string`, arrays, and any `class` instance are **reference types**: a variable holds a reference (pointer) to the data on the heap, and assigning it copies the reference, not the data.

Value type copy behaviour

```
int a = 10;
int b = a;   // b gets a COPY of a's value
b = 20;
Console.WriteLine(a);  // 10 — unaffected by change to b
```

Reference type sharing behaviour

```
int[] arr1 = { 1, 2, 3 };
int[] arr2 = arr1;     // arr2 points to the SAME array
arr2[0] = 99;
Console.WriteLine(arr1[0]); // 99 — arr1 sees the change too!
```

NOTE — This matches Java exactly
If you know Java: C#'s value types are like Java's primitives (`int`, `double`, etc.), and C#'s reference types behave like Java's objects and arrays. The mental model transfers directly. We'll revisit this in depth in Chapter 14 when we cover `struct` vs `class`.

3.4 Constants and Naming

const and readonly

```
const double Pi = 3.14159;   // must be set at compile time, never changes
Pi = 3.15;  // COMPILE ERROR
```

- **PascalCase** for class names, method names, and constants: `CalculateArea`, `MaxScore`.
- **camelCase** for local variables and method parameters: `studentName`, `totalMarks`.
- These are strong community conventions (enforced by style analyzers), similar to Java's camelCase conventions, but note C# capitalizes method names too (`ToString()`, not `toString()`).

3.5 Type Conversion

Implicit vs explicit conversion

```
int i = 100;
double d = i;         // implicit — no data loss possible, widening

double price = 99.99;
int whole = (int)price;   // explicit cast — required, narrowing, truncates to 99

string text = "123";
int parsed = int.Parse(text);          // throws if invalid
bool ok = int.TryParse(text, out int result); // safe version, no exception
```

WATCH OUT — Casting double to int truncates, it doesn't round
`(int)9.99` gives `9`, not `10`. If you need rounding, use `Math.Round(9.99)` first, then cast.

Practice Exercises — Chapter 3

1. Declare a `decimal` variable for a product price and a `double` for its weight in kg, then print both.

2. Predict the output: given `int x = 7; double y = x / 2;` what is `y`? Now fix the code so `y` correctly holds `3.5`.

3. Write a program with two `int` arrays. Assign one to the other, modify an element through the second variable, and print the first to prove they share memory.

4. Use `int.TryParse` to safely convert a string `"abc"` to an integer, and print a friendly message if it fails instead of crashing.

PART II

Core Language Fundamentals

Operators, control flow, methods, and data

CHAPTER 4

Operators & Expressions

Most C# operators are identical to what you already know from C/C++/Java. This chapter is intentionally brief on the familiar parts and focuses on what's new or different.

4.1 Arithmetic, Relational, Logical Operators

Category	Operators	Notes
Arithmetic	`+ - * / % ++ --`	Same as C/C++/Java
Relational	`== != > < >= <=`	Same as C/C++/Java
Logical	`&& || !`	Short-circuit, same as C/C++/Java
Assignment	`= += -= *= /= %=`	Same
Bitwise	`& | ^ ~ << >>`	Same

Integer division vs double division (same trap as C/C++/Java)

```
int a = 7, b = 2;
Console.WriteLine(a / b);        // 3 (integer division)
Console.WriteLine((double)a / b); // 3.5 (one operand promoted to double)
Console.WriteLine(a % b);        // 1 (modulo)
```

4.2 Null-Related Operators (New if you're not from C#)

C# has special operators for dealing with `null` cleanly — these don't exist in C/C++ and have no direct Java equivalent until fairly recent Java versions with `Optional`.

Null-coalescing ?? and null-conditional ?.

```
string name = null;
string display = name ?? "Guest";     // if name is null, use "Guest"
Console.WriteLine(display);            // Guest

string[] items = null;
int? count = items?.Length;            // ?. short-circuits to null
Console.WriteLine(count ?? 0);         // 0

name ??= "Default";                    // assign only if currently null
```

NOTE — Why this matters
In Java or C++, checking for null often means verbose `if (x != null) { ... }` chains, or in the worst case, a `NullPointerException`/segfault. C#'s `?.` and `??` let you write null-safe expressions in a single line. We'll go much deeper on null-safety in Chapter 19.

4.3 String Interpolation

String interpolation is C#'s answer to Python's f-strings, and it's used constantly — prefer it over string concatenation with `+`.

String interpolation vs concatenation

```
string name = "Subhajit";
int age = 21;

// Old style concatenation (avoid)
string msg1 = "Name: " + name + ", Age: " + age;

// Interpolated string (preferred)
string msg2 = $"Name: {name}, Age: {age}";

// Expressions and formatting inside {}
double price = 499.5;
string msg3 = $"Total: {price * 2:F2}";  // "Total: 999.00"
```

Coming from Python / C / C++ / Java
This is directly equivalent to Python's `f"Name: {name}, Age: {age}"`. Java has no built-in equivalent (you'd use `String.format` or `+`), so this is one of the nicer conveniences C# gives you.

4.4 The Ternary and is/as Operators

Ternary operator (same as C/C++/Java) and type-checking operators

```
int marks = 45;
string result = marks >= 40 ? "Pass" : "Fail";

object obj = "hello";
if (obj is string s)   // pattern-matching 'is'
{
    Console.WriteLine($"It's a string of length {s.Length}");
}
```

NOTE — Pattern-matching is preview
`obj is string s` both checks the type AND creates a new variable `s` of that type in one line. This is called pattern matching and we'll explore it fully in Chapter 20.

Practice Exercises — Chapter 4

1. Given `int total = 17, people = 5;`, compute and print the integer division and the remainder, then the precise decimal average using a cast.

2. Write a line using string interpolation that prints `"Result: 87.50%"` from a `double score = 87.5;` using a format specifier.

3. Use `??` to give a variable `string city = null;` a default value of `"Unknown"` and print it.

4. Write a ternary expression that prints `"Even"` or `"Odd"` for a given integer.

CHAPTER 5

Control Flow: Conditionals

Conditional statements in C# will feel almost identical to C/C++/Java, with one powerful addition: the switch expression.

5.1 if / else if / else

Standard if-else chain

```
int marks = 72;

if (marks >= 90)
{
    Console.WriteLine("Grade: A+");
}
else if (marks >= 75)
{
    Console.WriteLine("Grade: A");
}
else if (marks >= 40)
{
    Console.WriteLine("Grade: B");
}
else
{
    Console.WriteLine("Grade: F");
}
```

WATCH OUT — No truthy/falsy values
Unlike Python (`if x:`) or C (`if (x)` where any nonzero int is true), C#'s `if` condition must be a genuine `bool` expression. `if (count)` where `count` is an `int` will not compile.

5.2 switch Statement (Classic Form)

Classic switch — like Java/C, but no fallthrough allowed

```
int day = 3;
string name;

switch (day)
{
    case 1:
        name = "Monday";
        break;
    case 2:
        name = "Tuesday";
        break;
    case 3:
        name = "Wednesday";
        break;
    default:
        name = "Unknown";
        break;
}
Console.WriteLine(name);
```

WATCH OUT — break is mandatory, fallthrough is a compile error
In C/C++/Java, forgetting `break;` silently falls through to the next case — a classic bug source. C# makes implicit fallthrough a **compile-time error**. If you deliberately want fallthrough behaviour, you must use `goto case X;` explicitly.

5.3 switch Expression (Modern C#, 8.0+)

This is new and doesn't exist in C/C++/older Java. A switch **expression** directly produces a value, is far more compact, and supports pattern matching.

Modern switch expression

```
int day = 3;
string name = day switch
{
    1 => "Monday",
    2 => "Tuesday",
    3 => "Wednesday",
    4 => "Thursday",
    5 => "Friday",
    6 or 7 => "Weekend",
    _ => "Invalid"    // '_' is the default/discard pattern
};
Console.WriteLine(name);
```

Switch expression with pattern matching and conditions

```
int marks = 72;
string grade = marks switch
{
    >= 90 => "A+",
    >= 75 => "A",
    >= 40 => "B",
    _     => "F"
};
Console.WriteLine(grade);
```

NOTE — When to use which

Use the switch **expression** (`=>` form) whenever you're just computing/returning a value based on cases — it's shorter and safer. Use the classic switch **statement** when each branch needs to run multiple independent statements or side effects.

Practice Exercises — Chapter 5

1. Rewrite a grading if-else chain (A+/A/B/F based on marks) as a switch expression using range patterns.

2. Write a switch statement that prints the number of days in a given month number (1-12), handling February separately.

3. Use `or` patterns in a switch expression to classify a character as `'Vowel'` or `'Consonant'`.

CHAPTER 6

Control Flow: Loops

for, while, and do-while loops are essentially unchanged from C/C++/Java. The main new addition is the foreach loop, which you'll use constantly.

6.1 for, while, do-while

Standard loops — identical syntax to C/C++/Java

```
// for loop
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Count: {i}");
}

// while loop
int n = 5;
while (n > 0)
{
    Console.WriteLine(n);
    n--;
}

// do-while — body runs at least once
int x = 0;
do
{
    Console.WriteLine("Runs at least once");
} while (x > 0);
```

6.2 foreach — Iterating Collections

`foreach` is the idiomatic way to iterate over any collection (arrays, `List`, dictionaries, strings, etc.) in C#, and you'll use it far more often than an index-based `for`.

foreach over an array and a string

```
int[] scores = { 85, 90, 78, 92 };
foreach (int score in scores)
{
    Console.WriteLine(score);
}

foreach (char c in "Hello")
{
    Console.WriteLine(c);
}
```

Coming from Python / C / C++ / Java
This is the direct equivalent of Python's `for item in collection:` and Java's enhanced for-loop `for (int score : scores)`. Note C# uses the keyword `in`, matching Python, whereas Java uses `:`.

WATCH OUT — foreach variables are read-only
You cannot modify the collection's elements directly through the `foreach` loop variable (e.g. you can't do `score = 0;` inside the loop above to change the array). Use a regular indexed `for` loop if you need to mutate elements in place.

6.3 break, continue, and Labeled-Style Control

break and continue — same behaviour as C/C++/Java

```
for (int i = 1; i <= 10; i++)
{
    if (i == 5) continue;   // skip 5
    if (i == 8) break;      // stop entirely at 8
    Console.WriteLine(i);
}
```

NOTE — No labeled break/continue like Java
Java allows labeled loops (`outer: for(...) { break outer; }`) for breaking out of nested loops directly. C# does not have this — instead, use a `bool` flag, restructure into a method with an early `return`, or use `goto` (rare, generally discouraged).

Practice Exercises — Chapter 6

1. Write a `for` loop that prints the multiplication table of 7 (7x1 to 7x10).
2. Given `int[] marks = { 55, 91, 40, 23, 88 };`, use `foreach` to print only the passing marks (>= 40).
3. Write a program using `while` that keeps doubling a number starting from 1 until it exceeds 1000, printing each step.
4. Use nested `for` loops to print a right-angled triangle pattern of stars, 5 rows tall.

CHAPTER 7

Methods & Parameter Passing

C# calls functions "methods" because every function must belong to a class (there are no free-floating functions like in Python or C). Parameter passing has some genuinely new keywords worth learning carefully.

7.1 Defining and Calling Methods

Basic method syntax

```
class Program
{
    // returnType MethodName(parameters)
    static int Add(int a, int b)
    {
        return a + b;
    }

    static void Greet(string name)
    {
        Console.WriteLine($"Hello, {name}!");
    }

    static void Main()
    {
        int sum = Add(5, 7);
        Console.WriteLine(sum);   // 12
        Greet("Subhajit");
    }
}
```

A method that returns nothing uses `void` (same as C/C++/Java). Methods marked `static` belong to the class itself; without `static`, a method belongs to an **instance** of the class (we'll explore this fully in Chapter 10 on OOP).

7.2 Method Overloading

Multiple methods, same name, different parameters

```
static int Multiply(int a, int b) => a * b;
static double Multiply(double a, double b) => a * b;
static int Multiply(int a, int b, int c) => a * b * c;

// The compiler picks the right one based on argument types/count
```

Coming from Python / C / C++ / Java
Method overloading works exactly like in Java and C++. Python doesn't support this natively (a later `def` with the same name just overwrites the earlier one).

NOTE — Expression-bodied methods
`static int Multiply(int a, int b) => a * b;` is shorthand for a method whose body is a single return expression — equivalent to `{ return a * b; }`. You'll see this `=>` syntax often for short methods and properties.

7.3 ref, out, and in Parameters — New Concepts

By default, C# passes value types (int, double, struct, etc.) **by value** (a copy) and reference types (arrays, objects) **by reference to the object**, but the variable/pointer itself is still copied. To let a method modify the caller's actual variable, you need `ref` or `out`.

ref — pass by reference, variable must already be initialized

```
static void Double(ref int number)
{
    number *= 2;
}

static void Main()
{
    int value = 10;
    Double(ref value);
    Console.WriteLine(value);  // 20
}
```

out — method MUST assign it; used for multiple 'return values'

```
static bool TryDivide(int a, int b, out int result)
{
    if (b == 0)
    {
        result = 0;
        return false;
    }
    result = a / b;
    return true;
}

static void Main()
{
    if (TryDivide(10, 2, out int answer))
        Console.WriteLine(answer);  // 5
}
```

Keyword	Purpose	Caller variable must be initialized first?
(none)	Pass by value (copy)	Yes
`ref`	Pass by reference, can read AND write	Yes
`out`	Pass by reference, method must assign it (used for extra return values)	No
`in`	Pass by reference, but read-only inside the method (performance optimization for large structs)	Yes

> **Coming from Python / C / C++ / Java**
> C++ has `&` reference parameters which are similar to `ref`. Java and Python have neither — you cannot make a caller's primitive/local variable change through a plain method call in either language; this is genuinely new territory if you're coming from those.

7.4 Optional Parameters, Named Arguments, and params

Optional (default) parameters and named arguments

```
static void PrintInfo(string name, int age = 18, string city = "Unknown")
{
    Console.WriteLine($"{name}, {age}, {city}");
}

PrintInfo("Riya");                          // Riya, 18, Unknown
PrintInfo("Aman", 22);                      // Aman, 22, Unknown
PrintInfo("Dev", city: "Kolkata");          // named argument, skips age default
```

params — variable number of arguments

```
static int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int n in numbers) total += n;
    return total;
}

Sum(1, 2, 3);         // 6
Sum(1, 2, 3, 4, 5);   // 15
Sum();                // 0
```

Coming from Python / C / C++ / Java
`params` is directly equivalent to Java's varargs (`int... numbers`) and Python's `*args`. Named arguments work similarly to Python's keyword arguments.

Practice Exercises — Chapter 7

1. Write an overloaded `Area` method: one version for a rectangle (width, height) and one for a circle (radius).

2. Write a `Swap(ref int a, ref int b)` method that swaps two integers, and call it from Main to swap two variables.

3. Write a `TryParseAge(string input, out int age)` method that returns true/false and outputs the parsed age safely.

4. Write a `params`-based method `Average(params double[] values)` that returns the average of any number of doubles.

CHAPTER 8

Arrays & Strings

Arrays and strings are reference types in C# with rich built-in functionality, similar to Java's arrays and String class.

8.1 Arrays

Declaring, initializing, and accessing arrays

```
int[] numbers = new int[5];          // 5 zero-initialized ints
int[] scores = { 90, 85, 77, 100 };  // initializer shorthand
string[] names = new string[] { "Riya", "Aman", "Dev" };

Console.WriteLine(scores[0]);        // 90
Console.WriteLine(scores.Length);    // 4  (property, no parentheses!)

scores[1] = 88;                      // arrays are mutable
```

WATCH OUT — Length is a property, not a method
Unlike Java's `array.length` (field, no parens) mixed with `string.length()` (method, has parens), C# is fully consistent: `array.Length` and `string.Length` are BOTH properties — never write `Length()`.

Multi-dimensional and jagged arrays

```
// Rectangular 2D array (like a true matrix)
int[,] grid = new int[3, 3];
grid[0, 0] = 1;

// Jagged array — array of arrays, rows can differ in length
int[][] jagged = new int[3][];
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 1, 2, 3, 4 };
```

- `Array.Sort(scores)` — sorts in place.
- `Array.Reverse(scores)` — reverses in place.
- `Array.IndexOf(scores, 85)` — finds an element's index.
- Arrays have a **fixed size** once created — for a growable list, use `List` (Chapter 9).

8.2 Strings

`string` in C# is a reference type but behaves value-like for comparison, and is **immutable** — exactly like Java's `String` and Python's `str`. Every "modifying" method actually returns a brand-new string.

Common string operations

```
string s = "Hello, World!";

Console.WriteLine(s.Length);              // 13
Console.WriteLine(s.ToUpper());           // HELLO, WORLD!
Console.WriteLine(s.ToLower());           // hello, world!
Console.WriteLine(s.Substring(7));        // World!
Console.WriteLine(s.Substring(7, 5));     // World
Console.WriteLine(s.Replace("World", "C#")); // Hello, C#!
Console.WriteLine(s.Contains("World"));   // true
Console.WriteLine(s.IndexOf("World"));    // 7
Console.WriteLine(s.Trim());              // removes leading/trailing whitespace
Console.WriteLine(s.Split(", ")[1]);      // "World!"
```

String comparison — use == freely (unlike Java!)

```
string a = "hello";
string b = "hel" + "lo";
Console.WriteLine(a == b);          // true — value comparison
Console.WriteLine(a.Equals(b));     // true — also value comparison
```

Coming from Python / C / C++ / Java
This is a key difference from Java: in Java, `==` on Strings compares **references**, and you're taught to always use `.equals()`. In C#, the `==` operator is overloaded for `string` to always do a value comparison, so `==` is idiomatic and safe to use directly.

StringBuilder — for heavy string building

StringBuilder avoids creating many intermediate strings in loops

```
using System.Text;

StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++)
{
    sb.Append(i).Append(", ");
}
Console.WriteLine(sb.ToString());  // 1, 2, 3, 4, 5,
```

Normal string creates a new object on every modification, whereas StringBuilder modifies the same buffer, making repeated string operations faster and more memory-efficient.

NOTE — Same purpose as Java's StringBuilder
Because strings are immutable, `s += x` inside a loop creates a new string object every iteration — wasteful for large loops. `StringBuilder` mutates an internal buffer instead, just like Java's class of the same name.

Practice Exercises — Chapter 8

1. Create an `int[]` of 6 exam scores, find and print the highest score using `Array.Sort` (hint: sort then take the last element).

2. Given a full name string `"Subhajit Roy"`, split it and print the first name and last name on separate lines.

3. Write a program that checks whether a given string is a palindrome (reads the same forwards and backwards), ignoring case.

4. Use `StringBuilder` to build the string `"0-1-2-3-...-9"` inside a loop, then print the result.

CHAPTER 9

Collections: List, Dictionary & More

Arrays are fixed-size. For real applications you'll constantly reach for the generic collection types in `System.Collections.Generic` — the direct equivalent of Java's Collections Framework or Python's list/dict/set.

9.1 List — a Growable Array

List basics

```
using System.Collections.Generic;

List<string> names = new List<string>();
names.Add("Riya");
names.Add("Aman");
names.Add("Dev");

Console.WriteLine(names.Count);      // 3 (Count, not Length, for List!)
Console.WriteLine(names[1]);         // Aman — indexing works

names.Remove("Aman");                // remove by value
names.RemoveAt(0);                   // remove by index
names.Insert(0, "Priya");            // insert at position
Console.WriteLine(names.Contains("Dev")); // true

// Collection initializer syntax
List<int> scores = new List<int> { 90, 85, 77 };
foreach (int s in scores) Console.WriteLine(s);
```

WATCH OUT — Count vs Length — a common mix-up
`Array` uses `.Length`. `List`, `Dictionary`, and most other collections use `.Count`. Mixing these up is one of the most common beginner compile errors in C#.

Coming from Python / C / C++ / Java
`List` is the direct equivalent of Java's `ArrayList` and Python's built-in `list`. The `` syntax is a **generic type parameter** — same idea as Java generics (`ArrayList`). We cover writing your own generic types in Chapter 16.

9.2 Dictionary — Key-Value Pairs

Dictionary basics — like Java's HashMap or Python's dict

```
Dictionary<string, int> marks = new Dictionary<string, int>();
marks["Riya"] = 88;
marks["Aman"] = 76;
marks.Add("Dev", 91);       // alternative way to add

Console.WriteLine(marks["Riya"]);          // 88
Console.WriteLine(marks.ContainsKey("Sam")); // false

if (marks.TryGetValue("Aman", out int score))
{
    Console.WriteLine(score);   // 76 — safe lookup, no exception if missing
}

foreach (KeyValuePair<string, int> pair in marks)
{
    Console.WriteLine($"{pair.Key}: {pair.Value}");
}
```

WATCH OUT — Direct indexing throws if the key is missing
`marks["Unknown"]` throws a `KeyNotFoundException` if the key doesn't exist. Always use `ContainsKey` or `TryGetValue` when you're not sure a key is present — the same caution Python developers apply to plain `dict[key]` access versus `.get()`.

9.3 HashSet, Queue, Stack

Other common collections

```
// HashSet<T> — unique elements, no duplicates, O(1) lookup
HashSet<int> uniqueIds = new HashSet<int> { 1, 2, 3 };
uniqueIds.Add(2);          // ignored, already present
Console.WriteLine(uniqueIds.Count);  // 3

// Queue<T> — FIFO
Queue<string> line = new Queue<string>();
line.Enqueue("first");
line.Enqueue("second");
Console.WriteLine(line.Dequeue());  // first

// Stack<T> — LIFO
Stack<int> history = new Stack<int>();
history.Push(1);
history.Push(2);
Console.WriteLine(history.Pop());   // 2
```

C# type	Java equivalent	Python equivalent
`List`	`ArrayList`	`list`
`Dictionary`	`HashMap`	`dict`
`HashSet`	`HashSet`	`set`
`Queue`	`LinkedList` (as Deque)	`collections.deque`
`Stack`	`Deque`/`Stack`	`list` (as stack)

Practice Exercises — Chapter 9

1. Create a `List` of 10 numbers, then use a loop to remove all even numbers, printing the final list.
2. Build a `Dictionary` mapping 5 student names to their marks. Print the name with the highest mark (loop through and track the max).
3. Use a `HashSet` to find and print all unique numbers from an array that has duplicates: `{ 1, 2, 2, 3, 4, 4, 5 }`.
4. Simulate a browser "back button" history using a `Stack`: push 3 page URLs, then pop and print them in the order a back button would show.

PART III

Object-Oriented Programming

Classes, inheritance, interfaces, and encapsulation

CHAPTER 10

Classes & Objects

You already know OOP from Java/C++/Python's classes. This chapter maps that knowledge onto C#'s specific syntax, which is closest to Java's.

10.1 Defining a Class

A simple class

```
class Student
{
    public string Name;
    public int Age;

    public void Introduce()
    {
        Console.WriteLine($"Hi, I'm {Name}, age {Age}.");
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();
        s1.Name = "Subhajit";
        s1.Age = 21;
        s1.Introduce();   // Hi, I'm Subhajit, age 21.
    }
}
```

`new Student()` allocates the object on the heap and calls the constructor, exactly like `new Student()` in Java or `Student()` in Python (minus the explicit `new`). A `Student` variable holds a **reference** to that heap object — recall Chapter 3's reference type discussion.

10.2 Access Modifiers

Modifier	Meaning
`public`	Accessible from anywhere
`private`	Accessible only within the same class (default if omitted for members)
`protected`	Accessible within the class and its subclasses
`internal`	Accessible anywhere within the same project/assembly (no Java/C++ direct equivalent)
`protected internal`	Union of protected and internal
`private protected`	Intersection: subclasses, but only within the same assembly

Coming from Python / C / C++ / Java
`public`, `private`, and `protected` map directly to Java/C++. `internal` is new: it's C#'s way of scoping visibility to "this whole project/library", which Java approximates loosely with package-private (no modifier) access.

10.3 this Keyword

Using 'this' to disambiguate fields from parameters

```
class Student
{
    public string Name;

    public Student(string name)
    {
        this.Name = name;   // 'this.Name' is the field, 'name' is the parameter
    }
}
```

Coming from Python / C / C++ / Java
Identical usage and meaning to Java and C++'s `this`.

10.4 Objects Are References — Recap

Two variables, one object

```
Student a = new Student();
a.Name = "Original";

Student b = a;         // b points to the SAME object as a
b.Name = "Changed";

Console.WriteLine(a.Name);   // "Changed" — a and b share the same object
```

NOTE — Use == carefully with objects

Unlike `string`, the default `==` for a plain `class` compares **references** (identity), not field values — two separate `Student` objects with identical fields are NOT `==` equal by default. You can override this (Chapter 11 covers overriding `Equals`).

Practice Exercises — Chapter 10

1. Create a `Car` class with fields `Brand`, `Model`, and `Year`, and a method `Display()` that prints all three. Create two Car objects and display them.

2. Create a `BankAccount` class with a `private` field `balance` and `public` methods `Deposit(double amount)` and `Withdraw(double amount)` that respect the balance (no overdraft).

3. Demonstrate reference-sharing: create a `Point` class with `X` and `Y`, assign one variable to another, modify through the second, and print the first to show the shared state.

CHAPTER 11

Constructors, Properties & Encapsulation

Properties are C#'s standout feature for encapsulation — a much cleaner alternative to writing manual getter/setter methods everywhere, as you would in Java.

11.1 Constructors

Constructors, including constructor chaining with 'this(...)'

```
class Student
{
    public string Name;
    public int Age;

    public Student()  // parameterless (default) constructor
    {
        Name = "Unknown";
        Age = 0;
    }

    public Student(string name, int age)  // parameterized constructor
    {
        Name = name;
        Age = age;
    }

    public Student(string name) : this(name, 18)  // chains to constructor above
    {
    }
}
```

Coming from Python / C / C++ / Java
Constructor overloading works like Java/C++. The `: this(...)` chaining syntax is C#-specific — Java uses `this(...)` as the first statement inside the constructor body instead.

11.2 Properties — The C# Way to Encapsulate

In Java, you write `private int age;` plus `getAge()`/`setAge()` methods. C# has **properties**: syntax that looks like a public field to the caller, but is backed by get/set logic, giving you validation and encapsulation without verbose method calls.

Full property with backing field and validation

```
class Student
{
    private int age;   // backing field

    public int Age
    {
        get { return age; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Age cannot be negative");
            age = value;
        }
    }
}

Student s = new Student();
s.Age = 21;              // calls the 'set' block — looks like a field, isn't!
Console.WriteLine(s.Age); // calls the 'get' block
```

Auto-implemented properties — the common shorthand

```
class Student
{
    public string Name { get; set; }     // hidden backing field, auto-generated
    public int Age { get; set; }
    public string Id { get; }             // read-only after construction

    public Student(string id)
    {
        Id = id;   // allowed inside constructor even though there's no 'set'
    }
}
```

NOTE — This is the single most-used C# feature you haven't seen before
Almost every class you write in real C# code will use auto-properties (`{ get; set; }`) for its public data instead of plain fields. It gives you the flexibility to add validation logic later **without breaking calling code**, since callers always use `object.PropertyName` either way.

11.3 init and Read-Only Properties (Modern C#)

get → Read the value.
set → Change the value.
get; set; → Read and write (most common).
get; → Read-only. Value is usually set inside the constructor or by the class itself. Cannot be changed later.
get; init; → Value can be set only when creating the object (using an object initializer or constructor). Cannot be changed later.

init — settable only during object creation

```
class Student
{
    public string Name { get; init; }
    public int Age { get; init; }
}

Student s = new Student { Name = "Riya", Age = 20 };  // object initializer syntax
// s.Name = "Changed";   // COMPILE ERROR — init-only, can't change after creation
```

Coming from Python / C / C++ / Java

`init` has no equivalent in Java or C++ — it gives you immutability (like Python's `@dataclass(frozen=True)` or a `final` field set only in the constructor) while still allowing convenient object-initializer syntax at creation time.

Practice Exercises — Chapter 11

1. Rewrite your `BankAccount` class from Chapter 10 using an auto-property `Balance { get; private set; }` so external code can read but not directly modify the balance.

2. Create a `Rectangle` class with `Width` and `Height` properties, and a read-only property `Area` (get-only, computed as `Width * Height`).

3. Create a `Temperature` class with a `Celsius` property whose setter throws an exception if the value is below -273.15 (absolute zero). Test it with a valid and an invalid value.

4. Create an immutable `Point` class using `init` properties for `X` and `Y`, and construct one using object-initializer syntax.

CHAPTER 12

Inheritance & Polymorphism

C# supports single inheritance for classes (like Java), with virtual/override for polymorphism explicit and opt-in — a deliberate difference from Java's implicit-virtual model.

12.1 Basic Inheritance

A base class and a derived class

```
class Animal
{
    public string Name { get; set; }

    public void Eat()
    {
        Console.WriteLine($"{Name} is eating.");
    }
}

class Dog : Animal    // ':' means 'inherits from', unlike Java's 'extends'
{
    public void Bark()
    {
        Console.WriteLine($"{Name} says Woof!");
    }
}

Dog d = new Dog();
d.Name = "Tommy";   // inherited property
d.Eat();             // inherited method
d.Bark();            // Dog's own method
```

Coming from Python / C / C++ / Java
C# uses `class Dog : Animal` where Java uses `class Dog extends Animal`. The concept is identical: single inheritance only (no multiple class inheritance, unlike C++).

12.2 virtual, override, and base

This is a key difference from Java. In Java, every non-final, non-private method is implicitly overridable. In C#, a method is **not** overridable unless the base class explicitly marks it `virtual` — this is a deliberate design choice for safety and performance.

virtual / override — must be explicit

```
class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Some generic animal sound");
    }
}

class Cat : Animal
{
    public override void Speak()
    {
        base.Speak();               // also call the parent's version
        Console.WriteLine("Meow!");
    }
}

Animal a = new Cat();
a.Speak();
// Output:
// Some generic animal sound
// Meow!
```

WATCH OUT — Forgetting 'virtual' means no polymorphism
If `Speak()` in `Animal` is NOT marked `virtual`, then `Cat`'s version must use the `new` keyword instead of `override`, and calling `Speak()` through an `Animal`-typed reference will call `Animal`'s version, not `Cat`'s — this is called "method hiding", and it's a common source of confusion. Always use `virtual`/`override` when you intend real polymorphism.

Coming from Python / C / C++ / Java
`base.Method()` is the direct equivalent of Java's `super.method()` and C++'s `BaseClass::method()`.

12.3 sealed — Preventing Further Overriding/Inheritance

sealed class and sealed method

```
sealed class FinalUtility { }   // cannot be inherited (like Java's 'final')

class Cat : Animal
{
    public sealed override void Speak()   // can't be overridden further
    {
        Console.WriteLine("Meow!");
    }
}
```

12.4 Polymorphism in Practice

Storing different subtypes in a common list and calling overridden methods

```
List<Animal> animals = new List<Animal>
{
    new Cat(),
    new Dog(),
};

foreach (Animal a in animals)
{
    a.Speak();   // calls the RIGHT version for each actual object type
}
```

NOTE — Why this matters

This is the entire point of polymorphism: code written once (`a.Speak()`) automatically behaves correctly for every subtype, present or future, without any `if`/`switch` on type.

Practice Exercises — Chapter 12

1. Create a base class `Shape` with a `virtual` method `Area()` returning 0. Create `Circle` and `Square` subclasses that override `Area()` correctly.

2. Put several `Shape` objects (Circles and Squares) into a `List` and print each one's area using a `foreach` loop — verify polymorphism works.

3. Create a `Vehicle` base class with a `virtual` method `Description()`. Override it in a `Car` subclass so it calls `base.Description()` and then adds extra info.

4. Explain in one sentence what would go wrong if you forgot the `virtual` keyword on `Shape.Area()` above.

CHAPTER 13

Interfaces & Abstract Classes

Interfaces define a contract of behaviour without implementation; abstract classes are partway between a full class and an interface. Both concepts map closely to Java.

13.1 Interfaces

Defining and implementing an interface

```
interface IShape           // convention: interface names start with 'I'
{
    double Area();
    double Perimeter();
}

class Circle : IShape
{
    public double Radius { get; set; }

    public Circle(double radius) => Radius = radius;

    public double Area() => Math.PI * Radius * Radius;
    public double Perimeter() => 2 * Math.PI * Radius;
}
```

- A class can implement **multiple** interfaces (unlike single class inheritance): `class Circle : IShape, IComparable { ... }`
- Interface members are implicitly `public` and have no body (in classic usage) — only a signature.
- The `I` prefix naming convention (`IShape`, `IDisposable`, `IEnumerable`) is a strong C# community standard, unlike Java's typical unprefixed interface names.

Coming from Python / C / C++ / Java
This maps directly to Java's `interface` keyword and `implements`. C#'s single `:` is used for both class inheritance and interface implementation together: `class Dog : Animal, IPet`.

13.2 Default Interface Methods (C# 8+)

Interfaces can now provide default implementations

```
interface IGreetable
{
    string Name { get; }

    void Greet()   // default implementation, no override forced
    {
        Console.WriteLine($"Hello, {Name}!");
    }
}
```

NOTE — Newer C# feature, similar to Java 8's default methods
This lets library authors add new methods to an interface without breaking every existing implementation — directly analogous to Java 8's `default` interface methods.

13.3 Abstract Classes

abstract class — partial implementation, cannot be instantiated directly

```
abstract class Shape
{
    public string Color { get; set; }

    public abstract double Area();     // no body — subclasses MUST implement

    public void Describe()              // full, shared implementation
    {
        Console.WriteLine($"A {Color} shape with area {Area():F2}");
    }
}

class Square : Shape
{
    public double Side { get; set; }
    public override double Area() => Side * Side;
}

// Shape s = new Shape();   // COMPILE ERROR — cannot instantiate an abstract class
Square sq = new Square { Color = "Red", Side = 4 };
sq.Describe();   // A Red shape with area 16.00
```

Coming from Python / C / C++ / Java
Behaves exactly like Java's `abstract class` and C++'s pure-virtual-function classes.

13.4 Interface vs Abstract Class — When to Use Which

Question	Interface	Abstract class
Can it hold field state?	No (only properties/constants)	Yes
Can a class use more than one?	Yes, multiple interfaces	No, single inheritance only
Purpose	Defines a capability/contract ("can do X")	Defines a shared base identity ("is a X")
Constructor allowed?	No	Yes

NOTE — Rule of thumb
Use an **interface** when unrelated classes need to share a capability (e.g. `IComparable`, `ISavable`). Use an **abstract class** when classes share a common identity and some real, reusable implementation (e.g. `Shape`, `Employee`).

Practice Exercises — Chapter 13

1. Define an `IPayable` interface with a method `CalculatePay()`. Implement it in two unrelated classes, `Employee` and `Freelancer`, each computing pay differently.
2. Convert your `Shape`/`Circle`/`Square` classes from Chapter 12 to use an `abstract class Shape` with an abstract `Area()` method and a shared, non-abstract `PrintSummary()` method.
3. Create an interface `IMovable` with a default method `Stop()` that prints "Stopped", and a class `Car` implementing it that only defines `Move()`. Call both `Move()` and `Stop()` on a `Car` object.
4. Explain in your own words: why can't you write `new IShape()` even though `IShape` compiles fine?

PART IV

Intermediate C#

Value types, exceptions, generics, and functional-style code

CHAPTER 14

Structs, Enums & Value vs Reference Types Revisited

Now that you understand classes, we can properly explain struct — C#'s value-type alternative to class — and enums, which are more powerful in C# than in C or Java.

14.1 struct — Value-Type "Classes"

Defining and using a struct

```
struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    public double DistanceFromOrigin() => Math.Sqrt(X * X + Y * Y);
}

Point p1 = new Point(3, 4);
Point p2 = p1;      // COPIES the whole struct (value type!)
p2.X = 99;
Console.WriteLine(p1.X);  // 3 — p1 is untouched, unlike a class
```

A `struct` looks almost identical to a `class` — it can have fields, properties, methods, and constructors — but it is a **value type**. It's copied on assignment and passed by value into methods, just like `int` or `double`.

	class	struct
Type category	Reference type (heap)	Value type (usually stack)
Assignment	Copies the reference	Copies all the data
Inheritance	Supports full inheritance	Cannot inherit from another struct/class (can implement interfaces)
Default value	`null`	All fields zeroed, never null
Good for	Complex objects, entities with identity	Small, simple data bundles: Point, Color, Money

WATCH OUT — No struct equivalent in Java
Java has no direct equivalent to `struct` — every non-primitive type in Java is a reference type. C/C++ programmers will recognize `struct` immediately, though C#'s version also supports methods and constructors, unlike plain C structs.

NOTE — When to use struct vs class
Use `struct` for small, immutable-ish, short-lived data (a 2D point, an RGB color, a date range). Use `class` for everything else, especially anything with identity, mutable shared state, or that participates in inheritance. When in doubt, default to `class`.

14.2 Enums

Basic enum

```
enum Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }

Day today = Day.Wednesday;
Console.WriteLine(today);           // Wednesday (prints the NAME, not a number!)
Console.WriteLine((int)today);      // 2 — zero-based by default

if (today == Day.Saturday || today == Day.Sunday)
    Console.WriteLine("Weekend!");
else
    Console.WriteLine("Weekday.");
```

Enum with switch expression, and custom underlying values

```
enum HttpStatus
{
    Ok = 200,
    NotFound = 404,
    ServerError = 500
}

HttpStatus status = HttpStatus.NotFound;
string message = status switch
{
    HttpStatus.Ok => "Success",
    HttpStatus.NotFound => "Resource missing",
    HttpStatus.ServerError => "Something broke",
    _ => "Unknown"
};
Console.WriteLine(message);   // Resource missing
```

Coming from Python / C / C++ / Java
C# enums print their **name** by default (`Console.WriteLine(today)` shows "Wednesday"), unlike plain C enums (which are just named integers). This is closer to Java's enum, though C# enums are simpler — they can't have their own methods/fields the way Java enums can.

Practice Exercises — Chapter 14

1. Create a `struct Money` with fields `Rupees` and `Paise`. Demonstrate that copying a `Money` variable does not affect the original.

2. Create an `enum Season { Spring, Summer, Autumn, Winter }` and write a switch expression that returns a one-word description of typical weather for each.

3. Given `enum Direction { North, East, South, West }`, write a method `TurnRight(Direction d)` that returns the next direction clockwise (wrapping West back to North).

CHAPTER 15

Exception Handling

Exception handling in C# is close to Java's try/catch/finally, with a couple of small but important syntax differences (like C#'s single catch-all catch clause style).

15.1 try / catch / finally

Basic exception handling

```
try
{
    int[] numbers = { 1, 2, 3 };
    Console.WriteLine(numbers[5]);   // throws IndexOutOfRangeException
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
finally
{
    Console.WriteLine("This always runs, error or not.");
}
```

Catching multiple exception types

```
try
{
    int a = int.Parse("abc");   // throws FormatException
}
catch (FormatException ex)
{
    Console.WriteLine("Bad format: " + ex.Message);
}
catch (OverflowException ex)
{
    Console.WriteLine("Number too big: " + ex.Message);
}
catch (Exception ex)          // catch-all — always put LAST
{
    Console.WriteLine("Unexpected error: " + ex.Message);
}
```

WATCH OUT — No multi-catch in one clause like Java 7+
Java lets you write `catch (IOException | SQLException ex)`. C# requires a separate `catch` block per exception type (though you can share logic by calling a common helper method from each block).

15.2 Throwing Exceptions

Throwing built-in and custom exceptions

```
static void Withdraw(double balance, double amount)
{
    if (amount > balance)
        throw new InvalidOperationException("Insufficient funds");
    if (amount < 0)
        throw new ArgumentException("Amount cannot be negative", nameof(amount));
}
```

Custom exception class

```
class InsufficientFundsException : Exception
{
    public double ShortBy { get; }

    public InsufficientFundsException(double shortBy)
        : base($"Insufficient funds. Short by {shortBy:C}.")
    {
        ShortBy = shortBy;
    }
}

// Usage:
// throw new InsufficientFundsException(500);
```

Coming from Python / C / C++ / Java
Custom exceptions work exactly like Java: extend `Exception` (or a subclass), call `base(message)` in the constructor (Java: `super(message)`). Note C# has no **checked** exceptions — unlike Java, a C# method never has to declare `throws` in its signature, and callers are never forced by the compiler to catch anything.

15.3 using and Automatic Resource Cleanup

For objects that hold external resources (files, network connections, database handles), C# uses `using` blocks to guarantee cleanup, similar to Java's try-with-resources or Python's `with` statement.

using statement — automatically calls Dispose()

```
using (StreamReader reader = new StreamReader("data.txt"))
{
    string content = reader.ReadToEnd();
    Console.WriteLine(content);
}   // reader.Dispose() is called automatically here, even if an exception occurs

// Modern shorthand (C# 8+) — disposes at the end of the enclosing scope
using var writer = new StreamWriter("log.txt");
writer.WriteLine("Log entry");
```

Coming from Python / C / C++ / Java
This is directly equivalent to Java's `try (var reader = ...) { }` and Python's `with open(...) as f:`. Any type implementing `IDisposable` can be used this way.

Practice Exercises — Chapter 15

1. Write a `Divide(int a, int b)` method that catches `DivideByZeroException` and prints a friendly message instead of crashing.

2. Create a custom `InvalidAgeException` (extending `Exception`) and throw it from a `SetAge(int age)` method if age is negative or over 150.

3. Write a try/catch/finally block reading user input for a number (`int.Parse`), catching `FormatException`, and always printing "Done" in the finally block.

4. Research and name one .NET built-in exception type for each of: dividing by zero, accessing a null reference, and accessing an invalid array index.

CHAPTER 16

Generics

Generics let you write type-safe, reusable code that works with any type — the same concept as Java generics, and more capable than C++ templates in some ways while less powerful in others.

16.1 Why Generics?

The problem generics solve

```
// Without generics, you'd need a separate class per type,
// or use 'object' and lose type safety:
class IntBox
{
    public int Value;
}
class StringBox
{
    public string Value;
}
// Tedious and repetitive — generics fix this.
```

16.2 Generic Classes and Methods

A generic class

```
class Box<T>
{
    private T value;

    public void Set(T newValue) => value = newValue;
    public T Get() => value;
}

Box<int> intBox = new Box<int>();
intBox.Set(42);
Console.WriteLine(intBox.Get());   // 42

Box<string> strBox = new Box<string>();
strBox.Set("Hello");
Console.WriteLine(strBox.Get());   // Hello

// intBox.Set("text");   // COMPILE ERROR — type-safe!
```

A generic method

```
static T Max<T>(T a, T b) where T : IComparable<T>
{
    return a.CompareTo(b) > 0 ? a : b;
}

Console.WriteLine(Max(5, 10));           // 10
Console.WriteLine(Max("apple", "banana")); // banana
```

Coming from Python / C / C++ / Java
This is essentially identical to Java generics (`class Box`, `>`). C++'s templates are more powerful (compile-time duck typing, no constraints needed) but C#'s generics compile once and check constraints strictly, giving clearer error messages.

16.3 Generic Constraints (where)

Constraint	Meaning
`where T : class`	T must be a reference type
`where T : struct`	T must be a value type
`where T : new()`	T must have a public parameterless constructor
`where T : SomeBaseClass`	T must inherit from SomeBaseClass
`where T : ISomeInterface`	T must implement ISomeInterface

Multiple constraints combined

```
class Repository<T> where T : class, new()
{
    public T CreateNew() => new T();
}
```

16.4 Generics You Already Use: List and Dictionary

NOTE — Full circle
Every collection from Chapter 9 — `List`, `Dictionary`, `HashSet` — is itself a generic type from the .NET Base Class Library. Now you understand exactly how they achieve type safety for any element type.

Practice Exercises — Chapter 16

1. Write a generic `Pair` class holding two values of potentially different types, with a method `Show()` that prints both.

2. Write a generic method `Swap(ref T a, ref T b)` that swaps any two values of the same type, and test it with both `int` and `string`.

3. Write a generic class `Stack` from scratch (using a `List` internally) with `Push`, `Pop`, and `Peek` methods.

4. Write a generic method `FindMax(List items) where T : IComparable` that returns the largest item in a list.

CHAPTER 17

Delegates, Events & Lambda Expressions

This chapter covers genuinely new territory if you're coming from Java or Python: C#'s first-class support for treating methods as values, which underpins events, LINQ, and async programming.

17.1 Delegates — Type-Safe Function Pointers

A **delegate** is a type that represents a method signature. You can create a variable of a delegate type and assign any matching method to it, then call it indirectly.

Declaring and using a delegate

```
delegate int MathOperation(int a, int b);   // declares a delegate TYPE

static int Add(int a, int b) => a + b;
static int Multiply(int a, int b) => a * b;

static void Main()
{
    MathOperation op = Add;         // assign a method to the delegate
    Console.WriteLine(op(3, 4));    // 7

    op = Multiply;                  // reassign to a different method
    Console.WriteLine(op(3, 4));    // 12
}
```

Coming from Python / C / C++ / Java
This is similar in spirit to a C/C++ function pointer, or a Java functional interface (`Function`), but delegates are type-safe, can hold multiple methods (multicast), and are baked deeply into the language.

17.2 Lambda Expressions

Lambdas — anonymous inline functions

```
MathOperation add = (a, b) => a + b;
Console.WriteLine(add(3, 4));   // 7

Func<int, int, int> multiply = (a, b) => a * b;   // built-in generic delegate
Console.WriteLine(multiply(3, 4));   // 12

Action<string> print = message => Console.WriteLine(message);
print("Hello from a lambda!");

Predicate<int> isEven = n => n % 2 == 0;
Console.WriteLine(isEven(4));   // true
```

Built-in delegate type	Signature	Use
`Action`	no params, returns void	`Action greet = () => Console.WriteLine("Hi");`
`Action`	1+ params, returns void	`Action log = msg => Console.WriteLine(msg);`
`Func`	1+ params, returns TResult	`Func square = x => x * x;`
`Predicate`	1 param, returns bool	`Predicate isPos = x => x > 0;`

Coming from Python / C / C++ / Java
Lambdas are the C# equivalent of Python lambdas and Java lambdas (`(a, b) -> a + b`). `Func<>`, `Action<>`, and `Predicate<>` are C#'s built-in generic delegate types, saving you from declaring a custom `delegate` for common cases — roughly analogous to Java's `java.util.function` interfaces (`Function`, `Consumer`, `Predicate`).

17.3 Passing Methods as Arguments

A method that accepts a Func as a parameter

```
static List<int> FilterList(List<int> numbers, Func<int, bool> condition)
{
    List<int> result = new List<int>();
    foreach (int n in numbers)
    {
        if (condition(n))
            result.Add(n);
    }
    return result;
}

List<int> nums = new List<int> { 1, 2, 3, 4, 5, 6 };
List<int> evens = FilterList(nums, n => n % 2 == 0);
foreach (int n in evens) Console.WriteLine(n);   // 2, 4, 6
```

NOTE — This is the foundation of LINQ
The pattern above — a method that takes a lambda describing "what counts" — is exactly how LINQ's `Where`, `Select`, and friends work internally. We dedicate all of Chapter 18 to LINQ.

17.4 Events — Delegates for Notifications

An **event** is a delegate with restricted access: only the declaring class can trigger ("raise") it, but any outside code can subscribe a handler to it. This is C#'s built-in implementation of the Observer pattern.

Declaring and using an event

```
class Button
{
    public event Action Clicked;    // an event based on the Action delegate

    public void Click()
    {
        Console.WriteLine("Button was clicked.");
        Clicked?.Invoke();              // safely raise it, if anyone subscribed
    }
}

class Program
{
    static void Main()
    {
        Button btn = new Button();
        btn.Clicked += () => Console.WriteLine("Handler 1: logging click");
        btn.Clicked += () => Console.WriteLine("Handler 2: playing sound");

        btn.Click();
        // Output:
        // Button was clicked.
        // Handler 1: logging click
        // Handler 2: playing sound
    }
}
```

Coming from Python / C / C++ / Java

This maps to the Observer pattern you'd hand-roll with interfaces in Java (e.g. `addActionListener`), but events are a first-class, built-in language feature in C#, used heavily in UI frameworks (WinForms, WPF, MAUI) and general event-driven code.

Practice Exercises — Chapter 17

1. Declare a delegate `StringTransform` matching `string -> string`, assign two different lambdas to it (one that uppercases, one that reverses), and call both.

2. Write a method `Calculate(int a, int b, Func operation)` and call it three times with different lambda operations (add, subtract, multiply).

3. Write a generic method `List MyFilter(List items, Func predicate)` that reimplements basic filtering, and test it on a list of strings (keep only strings longer than 3 characters).

4. Create a simple `Alarm` class with an `event Action Rang` and a method `Ring()`. Subscribe two different lambda handlers and trigger it.

PART V

Advanced & Modern C#

LINQ, null-safety, pattern matching, and concurrency

CHAPTER 18

LINQ (Language Integrated Query)

LINQ is one of C#'s most beloved features: a unified, declarative way to query collections, databases, and XML using the same syntax. If you've used Python list comprehensions or Java Streams, you already understand the mindset — LINQ is more powerful than both.

18.1 Method Syntax — Where, Select, and Friends

Basic LINQ method-syntax chain

```
using System.Linq;

List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

var evenSquares = numbers
    .Where(n => n % 2 == 0)      // filter
    .Select(n => n * n)          // transform
    .ToList();                    // materialize into a List<int>

foreach (int n in evenSquares) Console.WriteLine(n);   // 4, 16, 36, 64, 100
```

Coming from Python / C / C++ / Java
`Where` = Python's `filter()` / list comprehension `if`. `Select` = Python's `map()` / list comprehension expression, and Java's `Stream.filter()`/`Stream.map()`. LINQ chains read very similarly to a Java Stream pipeline, but are baked into the language with query syntax too (below).

18.2 Common LINQ Methods

Method	Purpose	Example
`Where`	Filter elements	`list.Where(x => x > 5)`
`Select`	Transform each element	`list.Select(x => x * 2)`
`OrderBy` / `OrderByDescending`	Sort	`list.OrderBy(x => x.Age)`
`First` / `FirstOrDefault`	Get first match (throws / returns default)	`list.First(x => x > 5)`
`Any` / `All`	Boolean checks	`list.Any(x => x < 0)`
`Count`	Count matches	`list.Count(x => x % 2 == 0)`
`Sum` / `Average` / `Max` / `Min`	Aggregate	`list.Sum(x => x.Price)`
`GroupBy`	Group into buckets	`list.GroupBy(x => x.Category)`
`Distinct`	Remove duplicates	`list.Distinct()`
`ToList` / `ToArray`	Materialize the query	`query.ToList()`

A realistic example: student data

```
class Student
{
    public string Name { get; set; }
    public int Marks { get; set; }
}

List<Student> students = new List<Student>
{
    new Student { Name = "Riya", Marks = 88 },
    new Student { Name = "Aman", Marks = 45 },
    new Student { Name = "Dev",  Marks = 91 },
    new Student { Name = "Sam",  Marks = 30 },
};

var toppers = students
    .Where(s => s.Marks >= 40)
    .OrderByDescending(s => s.Marks)
    .Select(s => s.Name)
    .ToList();

foreach (string name in toppers) Console.WriteLine(name);  // Dev, Riya

double average = students.Average(s => s.Marks);
Console.WriteLine($"Class average: {average:F2}");
```

18.3 Query Syntax — SQL-Like Alternative

LINQ also offers a query-comprehension syntax that reads like SQL. It compiles down to exactly the same method calls as above — use whichever is more readable for a given query.

The same topper query, written as query syntax

```
var toppers2 =
    from s in students
    where s.Marks >= 40
    orderby s.Marks descending
    select s.Name;

foreach (string name in toppers2) Console.WriteLine(name);
```

NOTE — Deferred execution

LINQ queries are lazy by default — `var query = numbers.Where(...)` doesn't run anything yet, it just builds a query plan. The query only actually executes when you iterate it (`foreach`) or force it with `.ToList()`/`.ToArray()`/`.Count()` etc. This mirrors Python generators and Java Streams' laziness.

Practice Exercises — Chapter 18

1. Given `List nums = new List { 4, 9, 1, 15, 22, 3, 8 };`, use LINQ to print only the numbers greater than 5, sorted ascending.

2. Using the `Student` list from this chapter, use LINQ to find the single student with the highest marks (hint: `OrderByDescending(...).First()`).

3. Given a `List words`, use LINQ's `GroupBy` to group words by their first letter, then print each group and its members.

4. Use LINQ to check (with `Any`) whether any student in the list scored below 35, and print a pass/fail message for the whole class.

CHAPTER 19

Nullable Types & Modern Null-Safety

Null reference errors are one of the most common bugs in any language. C# gives you strong tools to catch them at compile time rather than at runtime — closer to what Kotlin offers than what Java traditionally did.

19.1 Value Types Can't Be Null — Until You Opt In

Nullable value types with ?

```
int age = null;       // COMPILE ERROR — int can never be null
int? age2 = null;     // OK — int? means 'Nullable<int>'

if (age2.HasValue)
{
    Console.WriteLine(age2.Value);
}
else
{
    Console.WriteLine("No age provided");
}

int safeAge = age2 ?? 0;   // fallback with null-coalescing operator (Chapter 4)
```

Coming from Python / C / C++ / Java
This solves the same problem as Java's `Integer` (nullable) vs `int` (not nullable) distinction, but is more explicit: `int?` is clearly `Nullable` under the hood, a real generic struct, not a separate boxed wrapper class.

19.2 Nullable Reference Types (C# 8+)

Reference types (`string`, `class` instances) could **always** be null historically — just like Java. Since C# 8, projects can enable **nullable reference type** checking, where the compiler tracks nullability as part of the type system and warns you at compile time if you might be about to dereference a null.

With nullable reference types enabled (default in new projects)

```
string name = null;          // WARNING: converting null to non-nullable type
string? nickname = null;     // OK — explicitly nullable, the '?' documents intent

void PrintLength(string s)   // s is NOT expected to be null
{
    Console.WriteLine(s.Length);   // compiler assumes this is safe
}

void PrintLengthSafe(string? s)  // s MIGHT be null — caller is warned
{
    Console.WriteLine(s?.Length ?? 0);  // must handle null explicitly
}
```

NOTE — This is a compile-time analysis, not a runtime guarantee
Nullable reference type warnings help you catch mistakes early, but the compiler can still be wrong in edge cases, and it's only warnings, not hard errors, by default. It's a huge improvement over having no signal at all (classic Java pre-`Optional`), but it doesn't fully eliminate `NullReferenceException` at runtime the way Rust's `Option` does.

19.3 The Null-Conditional Chain in Practice

Chaining ?. safely through several levels

```
class Address { public string City { get; set; } }
class Person { public Address HomeAddress { get; set; } }

Person p = new Person();   // HomeAddress is null

string city = p.HomeAddress?.City ?? "Unknown";
Console.WriteLine(city);   // "Unknown" — no crash, even though HomeAddress is null
```

Coming from Python / C / C++ / Java
Without `?.`, this would require a verbose `if (p.HomeAddress != null && p.HomeAddress.City != null)` check, exactly the kind of boilerplate Java developers write constantly before `Optional` or manual null checks.

Practice Exercises — Chapter 19

1. Write a method that takes an `int?` marks value and prints "Absent" if null, or the marks otherwise.
2. Create a `Person` class with a nullable `Address? HomeAddress` (where `Address` has a `string City`). Safely print the city or "Unknown" using `?.` and `??` for a person with no address.
3. Write a method `int? SafeDivide(int a, int b)` that returns `null` instead of throwing when `b` is 0, and demonstrate calling it with both a valid and a zero divisor.

CHAPTER 20

Pattern Matching & Records (Modern C#)

This chapter covers two of the most valuable modern C# features (introduced C# 7-9) that make code shorter, safer, and more expressive — features that go beyond what Java offered until very recently, and that Python and C++ still lack in this form.

20.1 Pattern Matching with is

Type patterns and property patterns

```
object shape = new Circle(radius: 5);

if (shape is Circle c && c.Radius > 3)
{
    Console.WriteLine($"Big circle, radius {c.Radius}");
}

// Property pattern — check nested fields directly
if (shape is Circle { Radius: > 10 })
{
    Console.WriteLine("Huge circle!");
}
```

20.2 Pattern Matching in switch Expressions

Type patterns combined in a switch expression

```
static string Describe(object obj) => obj switch
{
    int n when n < 0     => "Negative number",
    int n when n == 0    => "Zero",
    int n                => "Positive number",
    string s when s.Length == 0 => "Empty string",
    string s              => $"A string of length {s.Length}",
    null                   => "Nothing (null)",
    _                       => "Some other type"
};

Console.WriteLine(Describe(42));       // Positive number
Console.WriteLine(Describe("Hi"));     // A string of length 2
Console.WriteLine(Describe(null));     // Nothing (null)
```

Coming from Python / C / C++ / Java
This is far more expressive than a Java/C `switch` and even goes beyond Python's newer `match` statement in how tightly it integrates with the type system — matching on type, value ranges, and object shape all at once.

20.3 Records — Immutable Data Classes

A `record` is a special kind of class (or struct, with `record struct`) designed for immutable data. The compiler automatically generates value-based equality, a readable `ToString()`, and a concise "with" syntax for creating modified copies.

Defining and using a record

```
record Point(int X, int Y);   // auto-generates properties, equality, ToString

Point p1 = new Point(3, 4);
Point p2 = new Point(3, 4);

Console.WriteLine(p1 == p2);         // true — VALUE equality, unlike a class!
Console.WriteLine(p1);               // Point { X = 3, Y = 4 } — auto ToString

Point p3 = p1 with { Y = 10 };       // 'with' creates a modified COPY
Console.WriteLine(p3);               // Point { X = 3, Y = 10 }
Console.WriteLine(p1);               // Point { X = 3, Y = 4 } — original unchanged
```

	class	record
Equality (`==`)	Reference equality by default	Value equality automatically
`ToString()`	Type name only, by default	Readable, all-properties, automatically
Mutability	Mutable by default	Designed for immutability (`with` to "modify")
Best for	Entities with identity/behaviour	Data transfer objects, DTOs, immutable models

NOTE — Records are perfect for API models
If you go on to build a Web API with ASP.NET Core (Chapter 23), you'll define most of your request/response data shapes as `record` types — they're concise, safe by default, and compare/print sensibly, which is exactly what you want for data that flows in and out of an API.

Practice Exercises — Chapter 20

1. Write a `Describe(object obj)` switch expression that handles `int`, `double`, `bool`, and a default case, returning a description string for each.

2. Define a `record Student(string Name, int Marks);`. Create two students with identical data and prove they're `==` equal. Then use `with` to create a copy with different marks.

3. Write a property-pattern check: given a `record Rectangle(double Width, double Height)`, write an `if` using a property pattern that detects when it's a square (Width == Height).

CHAPTER 21

Async/Await & Basic Multithreading

C#'s async/await is one of the cleanest concurrency models in mainstream languages, and you'll need it constantly in .NET (every ASP.NET Core endpoint, every file/network call). It's conceptually similar to Python's asyncio, but far more integrated into the standard library.

21.1 Why Asynchronous Code?

Some operations (reading a file, calling a web API, querying a database) take time and don't need the CPU while waiting — they're waiting on I/O. Blocking a thread for that whole wait is wasteful, especially in a server handling many requests at once. `async`/`await` lets a method "pause" during the wait and free up the thread, without you manually managing callbacks.

21.2 async and await Basics

A simple async method

```
using System.Threading.Tasks;

static async Task<string> FetchDataAsync()
{
    Console.WriteLine("Starting fetch...");
    await Task.Delay(2000);         // simulates a 2-second I/O wait, non-blocking
    Console.WriteLine("Fetch complete.");
    return "some data";
}

static async Task Main()
{
    Console.WriteLine("Before call");
    string result = await FetchDataAsync();
    Console.WriteLine($"Got: {result}");
    Console.WriteLine("After call");
}
// Output order:
// Before call
// Starting fetch...
// (2 second pause, but the thread is FREE to do other work)
// Fetch complete.
// Got: some data
// After call
```

- A method marked `async` almost always returns `Task` (like `void`, but awaitable) or `Task` (like returning `T`, but awaitable).
- `await` pauses the current method until the awaited operation finishes, WITHOUT blocking the calling thread — that thread is released to do other work in the meantime.
- `await` can only be used inside a method marked `async`.

Coming from Python / C / C++ / Java
This directly parallels Python's `async def` / `await` with `asyncio`. It is NOT the same as starting a new OS thread (that's `Task.Run`, covered next) — `async`/`await` is about efficiently waiting for I/O, not about parallel CPU computation.

21.3 Real I/O Example: Calling a Web API

A realistic async HTTP call using HttpClient

```
using System.Net.Http;

static readonly HttpClient client = new HttpClient();

static async Task<string> GetWebPageAsync(string url)
{
    HttpResponseMessage response = await client.GetAsync(url);
    string content = await response.Content.ReadAsStringAsync();
    return content;
}

static async Task Main()
{
    string html = await GetWebPageAsync("https://example.com");
    Console.WriteLine(html.Length);
}
```

NOTE — The "Async" naming convention
By strong convention, every async method's name ends with `Async` (`GetAsync`, `ReadAsStringAsync`, `FetchDataAsync`). This isn't enforced by the compiler, but virtually every .NET library and codebase follows it — follow it too.

21.4 Task.Run — True Parallel Work

Task.Run for CPU-bound parallel work

```
static int SlowSquare(int n)
{
    Task.Delay(1000).Wait();   // simulate heavy CPU work
    return n * n;
}

static async Task Main()
{
    Task<int> t1 = Task.Run(() => SlowSquare(4));
    Task<int> t2 = Task.Run(() => SlowSquare(5));

    int[] results = await Task.WhenAll(t1, t2);   // run in parallel, wait for both
    Console.WriteLine($"{results[0]}, {results[1]}");   // 16, 25 — concurrent
}
```

NOTE — async/await vs Task.Run — pick the right tool

Use plain `async`/`await` for I/O-bound work (network, disk, database) — it scales to thousands of concurrent operations cheaply. Use `Task.Run` to push genuinely CPU-heavy work onto a background thread pool thread so it doesn't block your UI or main flow.

Practice Exercises — Chapter 21

1. Write an `async Task AddSlowlyAsync(int a, int b)` that awaits `Task.Delay(1000)` before returning `a + b`. Call and await it from `Main`, printing the result.

2. Use `Task.WhenAll` to run three separate `Task.Delay`-based async methods concurrently and print a message once all three have finished.

3. Explain in your own words why `await Task.Delay(2000)` does not freeze your whole program the way `Thread.Sleep(2000)` would block the current thread.

CHAPTER 22

File I/O & Working with Data

File handling in C# lives in System.IO, with both simple static helpers and stream-based classes for finer control — plus built-in JSON support you'll use constantly once you reach web APIs.

22.1 Reading and Writing Files — The Easy Way

File class static helpers — simplest approach

```
using System.IO;

// Write
File.WriteAllText("notes.txt", "Hello, file!");
File.AppendAllText("notes.txt", "\nAnother line.");

// Read
string content = File.ReadAllText("notes.txt");
Console.WriteLine(content);

string[] lines = File.ReadAllLines("notes.txt");
foreach (string line in lines) Console.WriteLine(line);

// Check existence, delete
if (File.Exists("notes.txt"))
{
    Console.WriteLine("File found.");
}
```

Coming from Python / C / C++ / Java
This is the C# equivalent of Python's `open(...).read()`/`.write()` shortcuts, or Java's `Files.readString`/`Files.writeString` (NIO). Great for simple, one-shot file operations.

22.2 StreamReader / StreamWriter — Finer Control

Reading line by line with a using block

```
using (StreamReader reader = new StreamReader("data.txt"))
{
    string line;
    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}

using (StreamWriter writer = new StreamWriter("output.txt", append: true))
{
    writer.WriteLine("New entry");
}
```

NOTE — Recall Chapter 15
`StreamReader`/`StreamWriter` implement `IDisposable`, so always wrap them in `using` (as shown) to guarantee the file handle is released, even if an exception occurs mid-read.

Async file I/O — combining Chapters 21 and 22

```
static async Task ReadFileAsync(string path)
{
    string content = await File.ReadAllTextAsync(path);
    Console.WriteLine(content);
}
```

22.3 Working with JSON

Modern .NET ships `System.Text.Json` built in — no external package needed for basic JSON work, unlike Java (which typically needs Jackson/Gson) or older .NET (which relied on Newtonsoft.Json).

Serializing and deserializing objects to/from JSON

```
using System.Text.Json;

record Student(string Name, int Marks);

Student s = new Student("Riya", 88);

string json = JsonSerializer.Serialize(s);
Console.WriteLine(json);   // {"Name":"Riya","Marks":88}

Student parsed = JsonSerializer.Deserialize<Student>(json);
Console.WriteLine(parsed.Name);   // Riya

// Writing/reading JSON straight to/from a file
File.WriteAllText("student.json", json);
string fromFile = File.ReadAllText("student.json");
Student reloaded = JsonSerializer.Deserialize<Student>(fromFile);
```

NOTE — This will be central once you reach ASP.NET Core
Web APIs communicate almost entirely in JSON. `JsonSerializer` (and the automatic (de)serialization ASP.NET Core does for you) is one of the most-used pieces of the entire .NET ecosystem.

Practice Exercises — Chapter 22

1. Write a program that writes 5 lines of student names to `students.txt`, then reads and prints them back using `StreamReader`.

2. Create a `record Book(string Title, string Author, int Year);`, serialize a list of 3 books to JSON, write it to a file, then read it back and deserialize it into a `List`.

3. Write a method that appends a new log line with a timestamp to `log.txt` every time it's called, using `File.AppendAllText`.

PART VI

Into the .NET Ecosystem

From language mastery to building real applications

CHAPTER 23

The .NET Ecosystem & Where to Go Next

You now know the C# language deeply enough to be productive. This final chapter is a map of the wider .NET ecosystem — the frameworks and tools built on top of C# — so you know what to learn next depending on what you want to build.

23.1 The .NET Project Types You'll Encounter

`dotnet new` template	What it builds
`console`	A command-line program (what you've used throughout this book)
`classlib`	A reusable library (a `.dll` with no entry point) referenced by other projects
`webapi`	A backend REST API using ASP.NET Core
`mvc`	A server-rendered web app (Model-View-Controller) using ASP.NET Core
`blazorserver` / `blazorwasm`	A web UI written in C# instead of JavaScript (Blazor)
`maui`	A cross-platform mobile/desktop app (Android, iOS, Windows, macOS)
`xunit` / `nunit`	A unit test project

23.2 ASP.NET Core — Building a Web API

ASP.NET Core is the framework you'll likely use most as a backend developer coming from Express.js. The mental model transfers directly: routes, request handlers, middleware, and JSON responses — just written in C#.

A minimal Web API — Program.cs (modern minimal API style)

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Hello from ASP.NET Core!");

app.MapGet("/students/{id}", (int id) =>
{
    return new { Id = id, Name = "Riya", Marks = 88 };   // auto-serialized to JSON
});

app.MapPost("/students", (Student s) =>
{
    Console.WriteLine($"Received: {s.Name}, {s.Marks}");
    return Results.Created($"/students/{s.Name}", s);
});

app.Run();

record Student(string Name, int Marks);
```

Coming from Python / C / C++ / Java
If you've used Express.js: `app.MapGet("/route", handler)` plays the same role as `app.get("/route", handler)`. Request bodies auto-bind to C# types (like `Student` above) the way `express.json()` middleware gives you `req.body` — except here it's strongly typed and validated automatically.

23.3 Other Key Pieces of the Ecosystem

- **Entity Framework Core (EF Core)** — an ORM, similar to Sequelize/Prisma (Node) or Hibernate (Java). Lets you work with a database using C# classes (`DbSet`) instead of raw SQL.
- **NuGet** — the package manager (`dotnet add package Newtonsoft.Json`), equivalent to npm.
- **xUnit / NUnit** — unit testing frameworks, similar to Jest (JS) or JUnit (Java).
- **Dependency Injection (built into ASP.NET Core)** — a pattern for wiring up services cleanly, conceptually similar to how NestJS handles DI in the Node world.
- **Blazor** — build interactive web UIs in C# instead of JavaScript/React, running via WebAssembly or a server connection.
- **MAUI (.NET Multi-platform App UI)** — one C# codebase targeting Android, iOS, Windows, and macOS — useful background for your React Native learning, as a native alternative.
- **Unity** — the game engine, which uses C# as its primary scripting language.

23.4 Suggested Learning Path From Here

- 1. Rebuild something you already made in Express.js (a small REST API) using ASP.NET Core minimal APIs — this is the fastest way to cement the mapping between what you know and C#.
- 2. Add EF Core with a SQLite or PostgreSQL database to that API for real persistence.

- 3. Learn dependency injection and the built-in `IServiceCollection` — core to any serious ASP.NET Core app.
- 4. Write a few unit tests with xUnit for your API's logic.
- 5. If mobile is your goal, explore .NET MAUI once your React Native project has given you a feel for cross-platform mobile UI concepts — a lot of the mental model (declarative-ish UI, platform abstraction) will carry over.

NOTE — You're ready
Everything in this book — types, OOP, generics, LINQ, async/await, records, pattern matching — is exactly what real-world ASP.NET Core codebases use daily. The jump from "knowing C#" to "building with .NET" is now mostly about learning the ASP.NET Core framework's conventions, not the language itself.

Practice Exercises — Chapter 23

1. Create a new project with `dotnet new webapi -o MyFirstApi`, run it, and hit its default endpoint in a browser or with curl.

2. Add a `GET /hello/{name}` minimal API endpoint that returns a personalized JSON greeting using the `name` route parameter.

3. Add a `POST /add` endpoint that accepts a JSON body `{ "a": 3, "b": 4 }` (define a `record AddRequest(int A, int B);`) and returns the sum.

4. Write down a 3-step personal plan for what you'll build next in .NET, based on the suggested learning path above.